

Like and Comment Functionality Implementation

Overview

This PR adds like and comment functionality to community posts (exercise submissions) in The Writer's Corner application.

Features Added

1. Like Functionality

- **Toggle Like/Unlike:** Users can like or unlike any public exercise submission
- **Like Counter:** Displays the number of likes each post has received
- **Visual Feedback:** Liked posts show a filled heart icon in rust color
- **Persistent Storage:** Likes are stored in the database with proper user-submission relationships

2. Comment Functionality

- **Add Comments:** Users can add comments to any public exercise submission
- **Edit Comments:** Users can edit their own comments
- **Delete Comments:** Users can delete their own comments
- **Comment Counter:** Displays the number of comments on each post
- **Real-time Updates:** Comment counts and lists update immediately after actions
- **Expandable Section:** Comments are shown in an expandable section to keep the UI clean

Technical Implementation

Database Schema Changes

New Models Added:

1. SubmissionLike

- `id` : Unique identifier
- `userId` : Reference to the user who liked
- `submissionId` : Reference to the exercise submission
- `createdAt` : Timestamp of when the like was created
- Unique constraint on `(userId, submissionId)` to prevent duplicate likes
- Index on `submissionId` for efficient queries

2. SubmissionComment

- `id` : Unique identifier
- `content` : The comment text
- `userId` : Reference to the user who commented
- `submissionId` : Reference to the exercise submission
- `createdAt` : Timestamp of when the comment was created
- `updatedAt` : Timestamp of last update
- Index on `submissionId` for efficient queries

Updated Models:

- **User:** Added relations to `submissionLikes` and `submissionComments`
- **ExerciseSubmission:** Added relations to `likes` and `comments`

API Routes

`/api/community/posts/likes`

- **POST:** Toggle like/unlike on a submission
- Request body: `{ submissionId: string }`
- Response: `{ liked: boolean, message: string }`
- Validates submission exists and is public
- Creates or deletes like based on current state
- **GET:** Get likes for a submission
- Query param: `submissionId`
- Response: `{ likes: Like[], count: number }`
- Includes user information for each like

`/api/community/posts/comments`

- **GET:** Fetch comments for a submission
- Query param: `submissionId`
- Response: `{ comments: Comment[] }`
- Includes user information and ordered by creation date
- **POST:** Add a new comment
- Request body: `{ submissionId: string, content: string }`
- Response: `{ comment: Comment }`
- Validates submission exists and is public
- Validates comment is not empty
- **PATCH:** Edit an existing comment
- Request body: `{ commentId: string, content: string }`
- Response: `{ comment: Comment }`
- Validates user owns the comment
- Validates comment is not empty
- **DELETE:** Delete a comment
- Query param: `commentId`
- Response: `{ message: string }`
- Validates user owns the comment

Updated `/api/community/posts`

- Now includes `likesCount`, `commentsCount`, and `isLikedByUser` for each post
- Efficiently fetches this data using Prisma's include feature

Frontend Components

Updated `community-overview.tsx`

- Added state management for:
- Expanded comments sections
- Comments data per post
- New comment input per post
- Comment editing state
- Loading states
- Added handler functions:
 - `handleLike` : Toggle like/unlike with optimistic UI updates
 - `fetchComments` : Fetch comments for a specific post
 - `toggleComments` : Show/hide comments section
 - `handleAddComment` : Add a new comment
 - `handleEditComment` : Edit an existing comment
 - `handleDeleteComment` : Delete a comment with confirmation
 - `getUserDisplayName` : Format user names consistently
 - `formatCommentDate` : Show relative time for comments
- UI Improvements:
 - Like button shows count and changes color when liked
 - Comment button shows count
 - Expandable comments section with smooth transitions
 - Comment input with “Post” button
 - Edit/Delete buttons for own comments (icon-based)
 - Inline editing for comments
 - Loading states for comments
 - Empty state messages

User Experience

Like Feature

1. Click the heart icon to like a post
2. Icon fills with color and count increments
3. Click again to unlike
4. Changes persist across page refreshes

Comment Feature

1. Click “Comment” button to expand comments section
2. View existing comments with author names and timestamps
3. Type in the input field and click “Post” or press Enter
4. Comments appear immediately in the list
5. Edit own comments using the edit icon
6. Delete own comments using the trash icon (with confirmation)

7. Relative timestamps (e.g., “2h ago”, “3d ago”)

Security & Validation

- All API routes require authentication
- Users can only edit/delete their own comments
- Submissions must be public to receive likes/comments
- Comment content is trimmed and validated for emptiness
- Proper error handling and user feedback
- Cascade deletes ensure data integrity

Database Migration

A migration file has been created at:

```
prisma/migrations/[timestamp]_add_likes_and_comments/migration.sql
```

To apply the migration:

```
cd app
npx prisma migrate deploy
```

Or for development:

```
cd app
npx prisma migrate dev
```

Testing Recommendations

1. Like Functionality

- Test liking and unliking posts
- Verify like counts update correctly
- Check that likes persist after page refresh
- Test with multiple users

2. Comment Functionality

- Test adding comments
- Test editing own comments
- Test deleting own comments
- Verify users cannot edit/delete others' comments
- Test empty comment validation
- Check comment ordering (oldest first)
- Test with multiple users

3. UI/UX

- Test expandable comments section
- Verify loading states
- Check responsive design
- Test keyboard interactions (Enter to submit)
- Verify confirmation dialog for delete

4. Edge Cases

- Test with deleted users
- Test with deleted submissions
- Test concurrent likes/unlikes
- Test very long comments
- Test special characters in comments

Future Enhancements

Potential improvements for future iterations:

- Reply to comments (nested comments)
- Like comments
- Mention users in comments (@username)
- Rich text formatting in comments
- Comment notifications
- Sort comments by newest/oldest
- Pagination for comments
- Report inappropriate comments
- Like animation effects
- Real-time updates using WebSockets

Files Modified

1. `app/prisma/schema.prisma` - Added SubmissionLike and SubmissionComment models
2. `app/app/api/community/posts/route.ts` - Updated to include likes and comments data
3. `app/app/api/community/posts/likes/route.ts` - New API route for likes
4. `app/app/api/community/posts/comments/route.ts` - New API route for comments
5. `app/components/community/community-overview.tsx` - Updated UI with like/comment functionality

Dependencies

No new dependencies were added. The implementation uses existing packages:

- Next.js API routes
- Prisma ORM
- NextAuth for authentication
- Existing UI components (Button, Input, Card, etc.)
- Lucide React icons (Heart, MessageCircle, Edit2, Trash2)